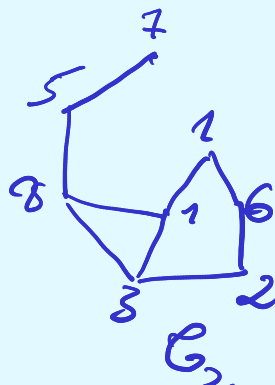
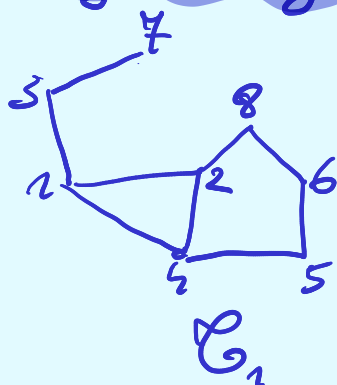


# AiSD

WYKŁAD XI

## SORTOWANIE-ZASTOSOWANIA

### izomorfizm grafów



Grafy  $G_1, G_2$  są izomorficzne, gdy (nieformalnie), gdy da się tak przetykietować wierzchołki  $G_2$ , żeby dostać  $G_1$ , formalnie

DEF  $G_1 = (V_1, E_1)$  izomorficzny z  $G_2 = (V_2, E_2)$

$\Leftrightarrow$

Istnieje permutacja  $\pi: V_2 \rightarrow V_1$ , że  $V_1 = \{\pi[v]: v \in V_2\}$   
 $E_1 = \{\{\pi[u], \pi[v]\}: \{u, v\} \in E_2\}$

Dla powyższego przykładu mamy  $\pi[8] = 1, \pi[1] = 2$ ,  
 $\pi[i] = i$  dla  $i \neq 8, i \neq 1$ .

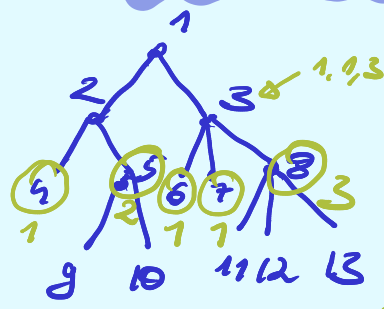
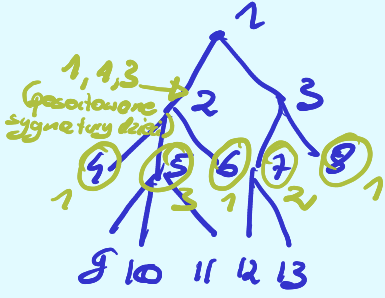
~~Problem jest NP-trudny.~~ Trudny, ale nie NP-Trudny (chyba).

### izomorfizm drzew

Wejście:  $T_1, T_2$  - dwa drzewa ekorenione

Wyjście: TAK/NIE zależnie od tego, czy drzewa są izomorficzne

W definicji izomorfizmu dla drzew ukoszonych,  
dodajemy dodatkowo, aby  $\pi(\text{root}(T_1)) = \text{root}(T_2)$



Na zistazjowo  
- sygnatury  
(sygnatury 2  
wiechołków  
są równe  $\Leftrightarrow$   
mają tyle samo  
dzieci)

Pomysł -  $\forall$  v policzyć głębokość

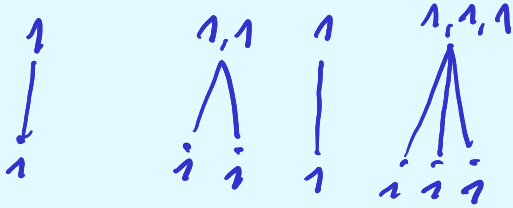
W listach przechowujemy wierzchołki tak, że  
w  $k$ -tej liście mamy wierzchołki z  $k$ -tego  
poziomu.

Signature:

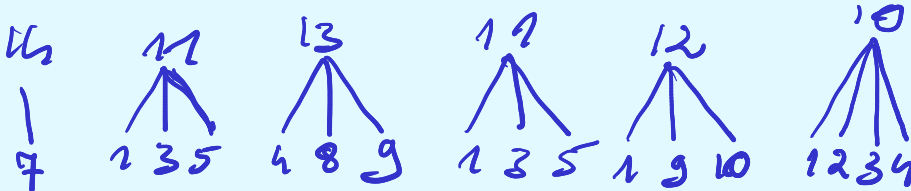
Die listi:

$i \quad i \ i \ i \quad i \ i \ i$

Dolej



Sortujemy cieżgi sygnatur  
dzieci. Jeżeli otrzymane ciężki  
są równe (dla 2 wierszółków),  
to przydzielamy tę samą  
sygnaturę, lub inną w podobnych  
przypadkach.

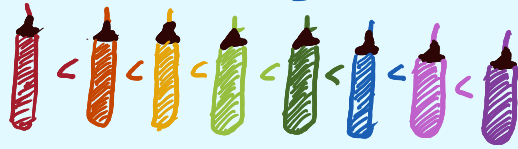


|      |   |    |
|------|---|----|
| 1234 | → | 10 |
| 135  | → | 11 |
| 135  | → | 11 |
| 1910 | → | 12 |
| 489  | → | 13 |
| 7    | → | 14 |

# Quick sort

mała ilustracja:-) Wiele razy było to tłumaczone na liściach, to teraz zrobimy na mazurkach.

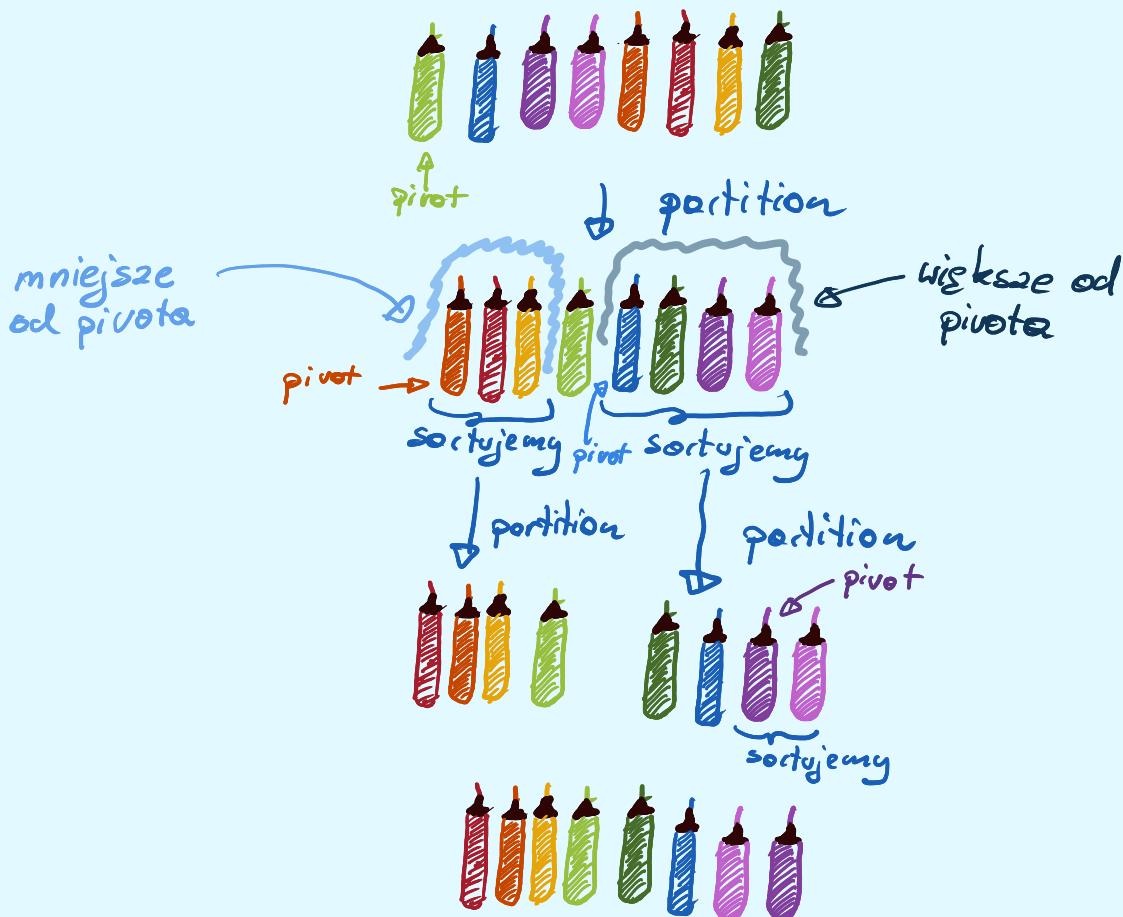
Porządek mazurków



W każdym wywołaniu wybieramy pivot i umieszczamy elementy mniejsze od niego na indeksach mniejszych, a większe na indeksach większych (partition).

Następnie sortujemy elementy na prawo i na lewo od pivotu.

TC - zależy od wyboru pivotu: Na razie wybieramy go jako pierwszy z brzoju.



## Pseudokod

QuickSort( $A[b \dots e]$ ):  
if ( $b < e$ ):  
     $q \leftarrow \text{Partition}(A[b, \dots, e])$        $\nearrow$  początek i koniec sortowanego fragmentu  
    QuickSort( $A[b, \dots, q-1]$ )  
    QuickSort( $A[q+1, e]$ )  
    ↓  
    podziel elementy względem  
    pewnego pivotu i zwróć  
    jego indeks

Partition można wykonać w miejscu. Szukamy kolejnych najmniejszych  $i \geq b$  i kolejnych największych  $j \leq e$ , t. że  $A[i] \geq p$  oraz  $A[j] < p$  i zamieniamy miejscami

Partition( $A[b, \dots, e]$ ):

$p \leftarrow$  dowolny element z  $A$  (pivot)

Poprawność pozostawiam  
jako ćwiczenie

$i \leftarrow b$   
 $j \leftarrow e$

while ( $j > i$ ):

    while  $A[i] < p$      $i++$

    while  $A[j] > p$      $j--$

    if  $j > i$ :

$A[i] \leftrightarrow A[j]$

$j--$   
         $i++$

return  $j$

UWAGA! Zwykłem wziąć z WDL

## TC:

Pivota wybieramy losowo z  $A[b \dots e]$  z rozkładem jednostajnym. Zakładamy, że elementy  $A$  są różne.

Niech  $a_1, \dots, a_n$  oznacza wartości z  $A$  posortowane rosnąco.

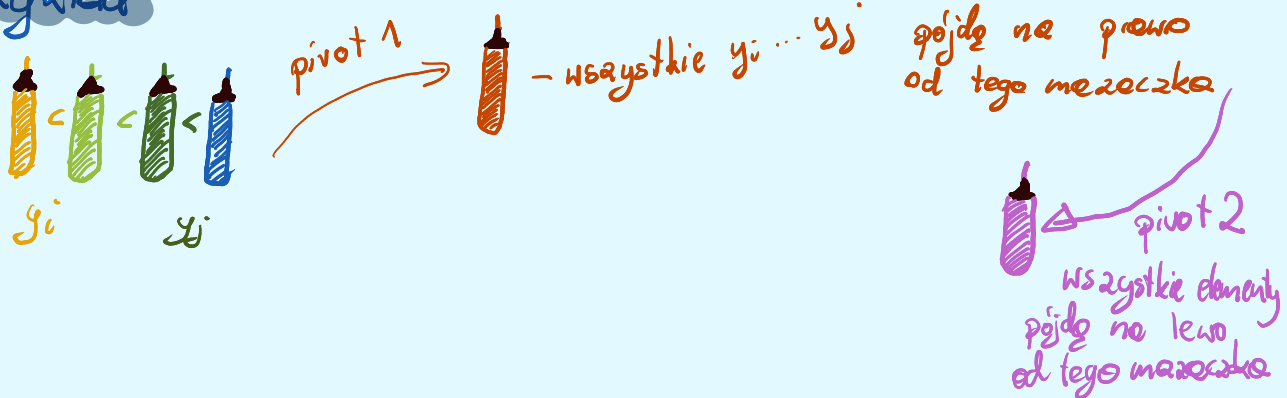
$X$  - liczba porównań

$$X_{ij} = \begin{cases} 1 & - a_i, a_j \text{ są porównywane} \\ 0 & \text{wpp} \end{cases}$$

$$\mathbb{P}(X_j = 1) = ?$$

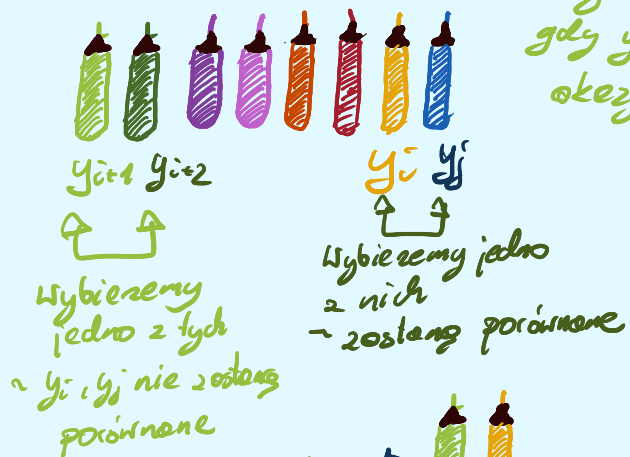
1° Dopóki żaden z elementów  $a_i^1, a_{i+1}^1, \dots, a_j^1$  nie stanie się pierwszym  $a_i^1, a_{i+1}^1, \dots, a_j^1$  nie będą porównywane ze sobą i będą się znajdować w tej samej podtablicy

# Paykital



2° Element  $y_i$  i  $y_j$  mogą zostać porównane jedynie kiedy jeden z nich jest pivotem. Co więcej mogą zostać jedynie porównane, gdy któryś z  $y_i \dots y_j$  został wybrany jako pivot. Jeśli wybierzemy pivotem jako  $y_i$  lub  $y_j$  to zostaną porównane, jeśli wybierzemy innego, to już nie zostaną porównane, bo trafią do różnych podtablic.

Przykład: Dochodzimy do momentu, gdy jeden z  $y_i \dots y_j$  zostaje pivotem. Jest to jedyny moment, gdy  $y_i \dots y_j$  mogą zostać porównane (jedyne okazyje, by któryś został pivotem).



Zatem szukane prawdopodobieństwo  
 jest równe temu, że  $y_i$  lub  $y_j$  stang  
 się pivotem w rozważonym momencie,  
 czyli  $\frac{1}{j-i+1} \cdot 2^k$  bo zarówno  $y_i$   
 jak  $y_j$  mogą być pivotem  
 tyle jest elementów  
 $y_i \dots y_j$   

$$= \frac{2}{j-i+1}$$

$y_i, y_j$  nie są  
 porównane

nosz zbiór  $y_1, \dots, y_j$   
 się podzielił  
 a szczególności  $y_i$  i  $y_j$   
 trafił do innych podzbiórów

*2. team*

X - liczba porównań

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] = 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{j-i+1} \leq 2 \sum_{i=1}^{n-1} (\log(n-i+1)) \leq 2 \sum_{i=1}^{n-1} O(\log n) \leq O(n \log(n))$$

# Analiza Fredmana

Rozważmy modyfikację quicksorta w której losujemy pivot tak długo, aż nie trafimy na takiego, który dzieli partycjonowaną tablicę tak, że każda podtablica ma długość co najmniej  $\frac{3}{4}$  długości pierwotnej.

Przykład  $n=8$   $\frac{3}{4} \cdot 8 = 6$

1 Proba

losujemy - dzieli w stosunku 0:7  
 $7 > \frac{3}{4} \cdot 8$  w tym wypadku to nie dobry mazarek: -[

2 Proba

losujemy - dzieli w stosunku 2:5  
 $2 < \frac{3}{4} \cdot 8$  oraz  
 $5 < \frac{3}{4} \cdot 8$   
 czyli to jest dobry mazarek: -)

Jakie jest ppb wylosowania takiego pivotu?



Zauważmy, że elementy  $\geq \frac{n}{4}$  co do wielkości i  $\leq \frac{3}{4}n$  co do wielkości będą dobrymi pivotami, czyli co najmniej  $\frac{3}{4}n - \frac{n}{4} = \frac{1}{2}n$  elementów są dobrymi pivotami. Czyli ppb wylosowania dobrego pivotu wynosi  $\geq \frac{1}{2}$ , oczekiwana ilość losowań  $\leq \sum_{i=0}^{\infty} 1 \cdot \left(\frac{1}{2}\right)^i = 2$ .

Po każdym losowaniu dokonujemy podziału i sprawdzamy ilość elementów na lewo i na prawo.

prawdopodobieństwo wystąpienia kolejnych prob

Czyli oczekiwany czas działania podziału wynosi  $2n$  (gdzie  $n$  to długość podtablicy)

SUKHAR

Jaka litera 'R' jest najbardziej wuu?



UWR:-)

Mamy zatem:

$$T(n) = T\left(\frac{1}{4}n\right) + T\left(\frac{3}{4}n\right) + 2n$$

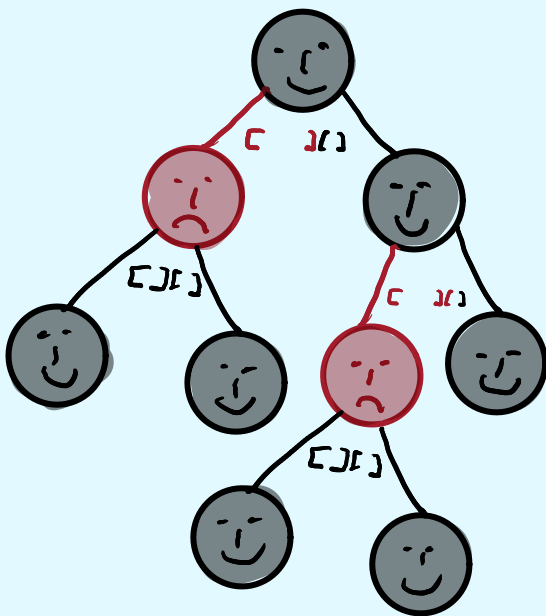
Maksymalna głębokość wynosi  $\log_{\frac{3}{2}} n = O(\log n)$

Na każdym poziomie wykonujemy  $2\left(\frac{1}{4}n + \frac{3}{4}n\right) = 2n$  operacji.

To daje złożoność  $2n \log n = O(n \log n)$ .

Przenosimy to rozumowanie na zrandomizowanego quicksorta.  
Zrobimy to na dwa sposoby.

Wizualizujemy wywołanie na drzewach (wierzchołki to podtablice)



😊 - czarny ma wielkość  $\leq \frac{3}{4}$  ojca, umiarnie - korzeń jest czarny

😞 - czerwony  $> \frac{3}{4}$  ojca

Mamy 3 obserwacje

1. Przy najmniej 1 syn jest czerwony (jeśli 1 jest niebieski, drugi ma  $< \frac{1}{2}n$  wierzchołków)

2. Czarnych poziomów jest  $\log n$  (najdłuższa ścieżka do liścia z wyciągniętymi czerwonymi wierzchołkami ma długość  $\log n$ , bo każdy kolejny czarny wierzchołek skracza podtablicę o  $\frac{3}{4}$ ).

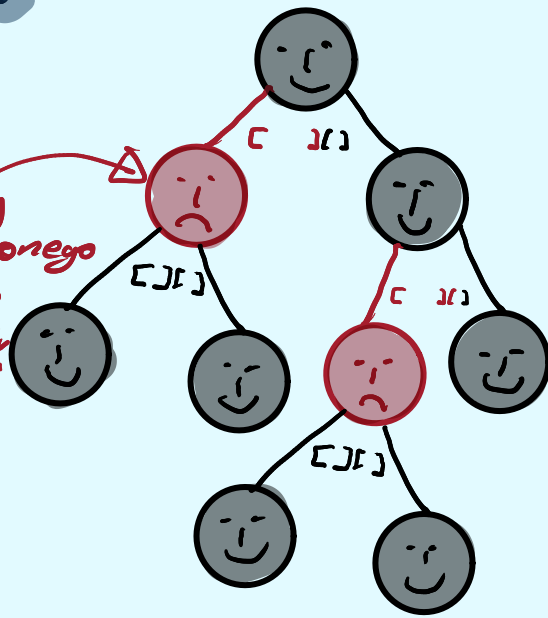
3. Oczekiwana długość ścieżki czerwonej wynosi 1.

Ozasadnienie: p.p.b., że jeden z synów ma tablicę długości  $\geq \frac{3}{4}$  ojcozskiej wynosi  $\frac{1}{2}$  (tyle ile wynosi p.p.b. wylosowania z tego pivota). P.b.p., że ścieżka taka ma długość i wynosi  $\left(\frac{1}{2}\right)^i$  (bo i rozgrywasz z tego pivota), zatem oczekiwana długość wynosi  $\sum_{i=1}^{\infty} \left(\frac{1}{2}\right)^i = 1$ .

Ile zatem wynosi oczekiwana wysokość? Zauważmy, że w najgorszym razie mamy na ścieżce na zmianę czerwony i czarny wierzchołek. Liczba poziomów czarnych wynosi  $\log n$ , dlatego ustawić pomiędzy nie czerwone uzyskamy wysokość  $\leq 2 \log n$ .

## Drugi sposób

interpretujemy obecność czerwonej wierzchołka, jako qudło w losowaniu pivotu w poprzedniej modyfikacji



$$\begin{bmatrix} b_1 \end{bmatrix} \begin{bmatrix} r_1 \end{bmatrix} \begin{bmatrix} r_2 \end{bmatrix}$$

pięć pierwszy  $\rightarrow [b_1][b_2]$   
dobry podział

gromadzimy  $b_1$  do  $b_4$  i  $\Gamma$

$$\Gamma: [b_1][b_2][b_3][b_4]$$

Podział  $A$  na tablice  
rozmiarów  $\leq \frac{3}{4}|A|$

- ~ Oczekiwana liczba podziałów do wystąpienia takiego podziału wynosi tyle samo ile qudło na wylosowanie pivotu
- ~ taki sam też poniesiemy koszt z powodu złych pivotów ( $2n$ ).

Dlatego TC quicksorta jest takie samo, jak przeróbki z losowaniem pivotu czyli  $O(n \log n)$ .

Źródło:

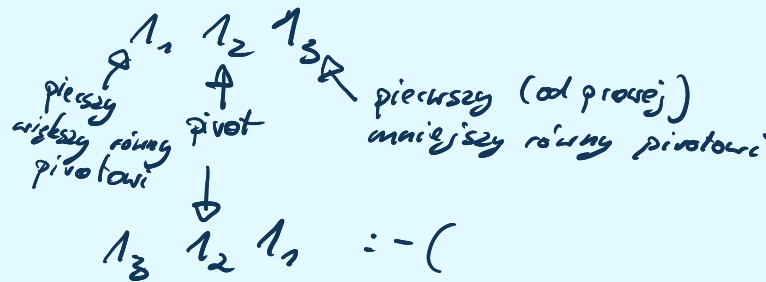
An intuitive and simple bounding argument for Quicksort

Michael L. Fredman

Rutgers University, New Brunswick, United States

Uwagi:

- ~ Quick sort jest **NIESTABILNY**.



- ~ Można zastosować magiczne pigułki (odsyłam do notatki o selekcji), ale ze względu na dużą stałą przy  $O(n \log n)$  w realnych zastosowaniach jest to niepraktyczne i randomizacja daje lepsze rezultaty.

- ~ standardowo gdy  $b$ -e jest małe stosujemy insert-sort - by zbić liczbę wywołań rekurencyjnych.